Laboratoire AAM O93_0 – Audit, expériences et validation par phases

AAM – dossier de conception et de preuve

13 octobre 2026

Résumé

Ce laboratoire établit la base traçable de O93_0 à partir de `sources_AAM_17.zip`. Il prend la théorie des empreintes comme fondement conceptuel, O93 et O92_0 comme sources logicielles récentes et les modèles historiques comme contexte. Il transforme les besoins en expériences, critères de sortie, contrats et preuves. Il ne prédit ni réussite ni échec ; une capacité décrite, exécutée, testée et acceptée par test indépendant relève de quatre niveaux de preuve différents.

Table des matières

1 Description et sources **3**
- 1.1 Mission . . . 3
- 1.2 Ordre et importance des sources . . . 3
- 1.3 Constats source O93 . . . 3

2 Architecture de laboratoire **5**
- 2.1 Graphe de fonctionnement . . . 5
- 2.2 Deux modes . . . 5
- 2.3 Format invariant des rapports . . . 5

3 Expériences abstraites **6**
- 3.1 A – identité observationnelle . . . 6
- 3.2 B – génération de sondes . . . 6
- 3.3 C – espaces sémantiques . . . 6
- 3.4 D – lecture de la demande . . . 6
- 3.5 E – autonomie causale . . . 7
- 3.6 F – observer / reproduire . . . 7

4 Plan de phases et critères de sortie **8**

5 Commandes de référence **9**

6 Conséquences et statut **10**
- 6.1 Théoriques . . . 10
- 6.2 Conceptuelles . . . 10
- 6.3 Opérationnelles . . . 10
- 6.4 Conclusion/statut . . . 10

7 Dossier de validation P7 – lecteurs multimodaux réels **11**
- 7.1 Description . . . 11
- 7.2 Architecture . . . 11
- 7.3 Pipeline complet . . . 11
- 7.4 Contrats de statut . . . 12
- 7.5 Commandes exactes d'exécution . . . 12
- 7.6 Environnement et intégrité . . . 12
- 7.7 Résultats bruts . . . 13
- 7.8 Interprétation des résultats . . . 13
- 7.9 Conséquences . . . 13
- 7.10 Erreurs, limites et conclusion/statut . . . 13
- 7.11 Anomalies documentaires et retests . . . 13
- 7.12 Réconciliation finale de P7 – lecteurs, entraînement et preuves . . . 14
  - 7.12.1 Description et architecture . . . 14
  - 7.12.2 Pipeline exécuté . . . 14
  - 7.12.3 Commandes de référence réellement exécutées . . . 14

7.12.4 Résultats bruts 14
7.12.5 Interprétation des résultats 14
7.12.6 Conséquences théoriques, conceptuelles et opérationnelles 15
7.12.7 Erreurs, limites et statut 15

Chapitre 1

Description et sources

1.1 Mission

Construire un AAM unifié qui observe les données brutes et leurs vues, crée des sondes différenciées, cherche invariants/variantes, construit des empreintes et des espaces sémantiques, apprend des mécanismes reproductibles, les compose et les adapte, puis mémorise les conséquences dans une boucle autonome persistante. Le laboratoire établit les faits de départ, les questions à tester, les critères d'acceptation, les erreurs et le statut effectif.

1.2 Ordre et importance des sources

Source	Fonction
`DDD_Base_theorie_des_empreintes_complete_order_5.tex` (racine et `theorie_emp`)	Source principale ; ordre 5, observation, empreintes, famille de sondes, équivalence observationnelle, distances/résolutions, contraintes, profils, généralisation et frontières de preuve.
`theorie_emp/th_emp_observation_formalisme_0(1).tex` et formalismes DD_Dev/empirical	Compléments théoriques et formalisme opérationnel d'observation.
`formalisme_O93.tex`, `laboratoire_O93.tex`, code, tests, logs O93	Source logiciel prioritaire : runtime commun, observateur, sondes adaptatives, compétences, acquisition source-off, RAG observation-first, mémoire, autonomie et sécurité.
`formalisme_model_emergent_O92.tex`, formalismes/laboratoire O92_0	Héritage, méthodes d'apprentissage et contexte des corrections.
`dataset/`	SMS, jeux JSONL, Parquet et météo. Étiquettes seulement dans des expériences explicitement supervisées.
`formalismes/` et `rapports/`	Historique des modèles et des expériences ; ils n'équivalent pas à une preuve de la version courante.

L'archive racine présente 220 entrées listées (dont entrées de répertoire) ; les lots O93 annoncent respectivement 504 entrées Dmodel, 181 Amodel et 413 suivi. L'inventaire exécuté et les SHA-256 sont fournis dans `source_inventory.json`. L'archive O92_0 et son patch sont conservés dans l'audit.

1.3 Constats source O93

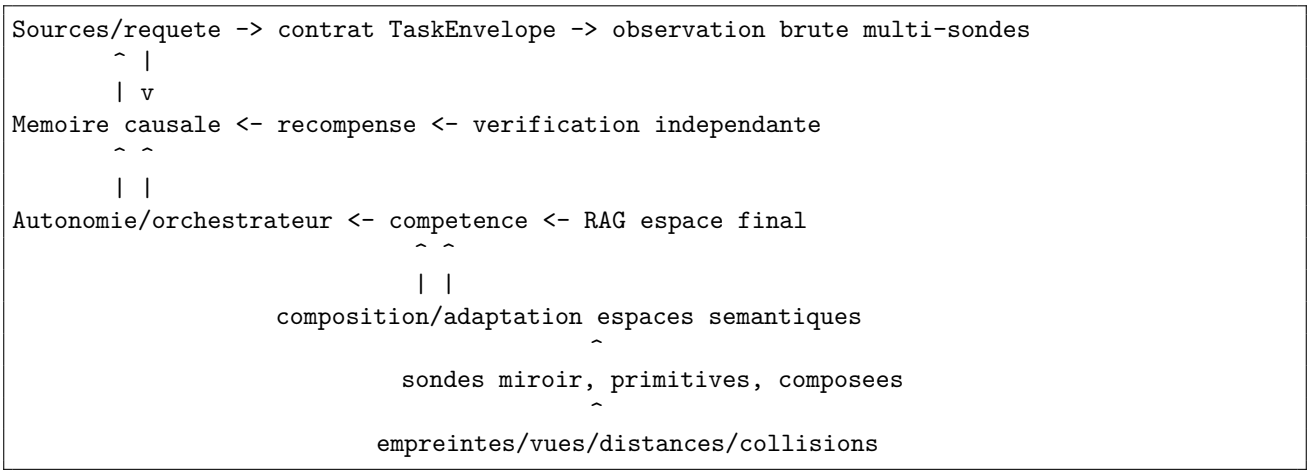
Les modules hérités exécutent déjà l'observation, les probes, l'acquisition de compétences, une mémoire, une boucle d'autonomie, une recherche lexicale et une interface Tk. Les journaux O93 déclarent P0–P9 réussies dans leur protocole antérieur, tandis que P10 reste bloquée sur des tests Windows natifs, matériels physiques et fournisseurs Internet indisponibles. Ces succès historiques ne valident pas les exigences nouvelles O93_0.

1. Le RAG hérité stocke des morceaux observés, mais la recherche de contenu hérité est lexicalement indexée. Il manque encore la chaîne sémantique demandée : espaces distincts, classes itoken/vtoken, distances, évolution de poids, espace final et lecteur pondéré.
2. Les probes ont un mode de recherche adaptative bornée. Un candidat trouvé au développement n'est pas une preuve de reproduction, de transfert ni de généralisation.
3. Le registre des compétences existe ; l'interface détaillée pour gestion complète, sélection manuelle, arbres de composition et réglages de mémoire reste à démontrer.
4. L'interface fournie expose cinq espaces principaux et ne couvre pas visiblement tout le schéma souhaité : historique latéral, inspection d'espaces, entraînement détaillé, internet, graphe de mémoire et focus.
5. La boucle autonome hérité démarre avec l'UI ou une commande ; le démarrage automatique de toute exécution applicative reste à concevoir et tester. Les init CLI ponctuelles doivent rester capables de s'arrêter proprement.
6. L'observation multimodale peut ne donner que des propriétés structurelles. Le code doit distinguer explicitement contenu décodé, partiel, inconnu et non exécuté.

Chapitre 2

Architecture de laboratoire

2.1 Graphe de fonctionnement



2.2 Deux modes

Le mode local doit fonctionner sans réseau, capteur ou fournisseur externe : UI, mémoire, observation locale, compétences présentes, entraînement de proximité et RAG. Le potentiel étendu ajoute les ressources autorisées (accélération, entraînement large, connecteurs Internet et matériel), sans supprimer le mode local. Un défaut d'environnement produit un statut explicite, pas un jugement de potentiel.

2.3 Format invariant des rapports

Chaque rapport, sans exception, comporte : description ; architecture ; pipeline complet ; commandes exactes ; environnement, jeux de données, seeds et hashes ; résultats bruts ; interprétation ; conséquences théoriques, conceptuelles et opérationnelles ; erreurs/limites ; conclusion ; statut du modèle et gates suivantes. Un test non exécuté est noté `NOT_RUN`.

Chapitre 3

Expériences abstraites

3.1 A – identité observationnelle

Construire des objets avec des transformations distinctes (ordre, casse, ponctuation, champs, miroir spatial/temporel lorsque les décodeurs sont disponibles). Calculer séparément le hash brut et les vues typées. Mesurer collision et fausse séparation au regard de critères de tâche explicites. Résultats : sorties de chaque sonde, paramètres, durée, hash et erreurs. Interprétation : une empreinte n'affirme qu'une équivalence relative à sa famille de sondes. Conséquences théoriques : distinguer égalité d'octets et quotient observationnel ; conceptuelles : profils de collisions ; opérationnelles : demander de nouvelles sondes si les vues sont insuffisantes. Statut : ne pas conclure universellement d'un essai jouet.

3.2 B – génération de sondes

Mesurer sondes primitives, paramètres, miroirs et compositions. Chaque sonde est identifiée par un hash de définition canonique, exécutée sans consulter la cible, puis évaluée séparément. Suivre gain marginal, distances, collisions, fausses séparations, stabilité et coût. Résultats : portefeuille, matrice de différences, traces et checkpoint. Interprétation : plus de sondes ne garantit pas de meilleure généralisation. Conséquences : continuer tant que l'information gagne et le budget le permet ; si le budget finit, reprise obligatoire. Le statut `CANDIDATE_FOUND_NOT_GENERALIZATION_PROOF` ne vaut pas acceptation.

3.3 C – espaces sémantiques

Pour chaque sonde, construire un espace indépendant contenant tokens, occurrences, relations contextuelles, classe, poids et provenance. Pour des vecteurs valides : $s(t,u) = \max(0, \min(1, \cos(z_t, z_u)))$ et $d(t,u) = 1 - s(t,u)$. Si I_j est l'ensemble des invariants de l'espace j, $p_j(t) = \max_{u \in I_j} s_j(t,u)$. La mise à jour vtoken doit croître monotoniquement en $p_j(t)$; les itokens sont pondérés par stabilité et proximité aux autres itokens. Le nombre d'espaces s'adapte à la variation des ensembles/rangs/poids, à la stabilité de lecture et au gain marginal, avec statut de convergence explicite. Résultats : hashes des sources et des espaces, itoken/vtoken, distances, poids, score de lecture. Interprétation : convergence heuristique n'est pas preuve de compréhension générale. Conséquences : un RAG inspectable et régénérable. Statut initial : la fonctionnalité nouvelle n'est pas prouvée par le RAG lexical hérité.

3.4 D – lecture de la demande

Observer la requête par plusieurs sondes du même type que celles du RAG ; construire son contexte ; classer/retrouver par poids, type itoken/vtoken, distance et proximité contextuelle ; garder les citations et la provenance ; router automatiquement ou manuellement les compétences. La recherche lexicale reste une compétence distincte. Résultats : contribution des tokens, passages sourcés, compétence

choisie et lacunes. Interprétation : similarité ne remplace pas une preuve de vérité. Statut : ajouter tests d'ablation des poids et de recherche lexicale.

3.5 E – autonomie causale

Démarrer AAM sans données puis observer les pulses, mémoire, agenda et erreurs. Soumettre une requête et vérifier sa priorité, puis reprise de l'auto-observation. Chaque mise à jour mémorielle relie cause, observation, décision, effet, score, parent et hash. L'enregistrement de raisonnement détaillé peut être stoppé séparément de la mémoire de contrôle. Le feedback utilisateur guide le diagnostic mais ne remplace pas un vérificateur indépendant. Statut : tests de concurrence, pause, arrêt, reprise et crash requis.

3.6 F – observer / reproduire

Charger un fichier et, après consentement explicite, un périphérique ; préserver l'entrée brute ; générer les sondes de façon itérative ; comparer les sorties ; enregistrer les statuts de décodage et tenter une reproduction source-off. Aucun code ne doit retourner une sortie préprogrammée pour réussir le test. Un périphérique absent ou un format inconnu ne devient jamais un succès fictif. Statut : séparer tests de fichiers, simulation abstraite et matériel réel.

Chapitre 4

Plan de phases et critères de sortie

Phase	But	Gate / preuves requises	État initial
P0	Audit et base	Inventaire haché, documents relus, contrats/statuts, checklists, registre, préflight, tests fondation et rapport reproductible	À mesurer
P1	Observation universelle	Texte/structures/médias testés ; source inchangée ; provenance ; vues ; statuts décodage ; capteurs sous consentement	À mesurer
P2	Moteur de sondes	Unicité, miroirs/composition, distances, collisions, continuation/checkpoint et aucun oracle en génération	À mesurer
P3	RAG sémantique	Espaces, itoken/vtoken, poids/distances, convergence, final, lecteur, nettoyage et tests d'ablation	À mesurer
P4	Compétences	Compétences de base justifiées, sélection auto/manuelle, composition, reproduction source-off et généralisation indépendante	À mesurer
P5	Autonomie/ mémoire	Autostart, priorité requête, focus, causalité, récompense, mémoire ON/OFF, pause/stop/reprise	À mesurer
P6	UI intégrée	Chat/historique, Observer, RAG, sondes, compétences, mémoire, entraînement, Internet, réglages ; contrôles connectés et testés	À mesurer
P7	Données/ entraînement	lecteurs multi-formats, champs facultatifs selon méthode, ajout/mise à jour RAG, pause, arrêt, suivi	À mesurer
P8	Intégration/sécurité	Tests bout-en-bout, permissions, erreurs, intégrité, non-régression et récupération	À mesurer
P9	Livraison portable	extraction propre, self-check, manifestes, lanceurs, guide, preuves synchronisées	À mesurer
P10	Acceptance externe	Windows natif, matériels physiques, Internet configuré, crash/rollback/ACL et rapports	À mesurer

Avant toute exécution, lire le présent formalisme, la checklist de phase et les deux registres d'erreurs. Après, journaliser commande, code retour, sorties et erreurs, mettre à jour les checklists, puis réexécuter la phase affectée. Une phase dépendante est rouverte lorsque le code, les documents ou les paramètres pertinents changent.

Chapitre 5

Commandes de référence

```
python -m aam_o93 status
python -m aam_o93 observe "path/to/file.pdf"
python -m aam_o93 knowledge-ingest "path/to/corpus"
python -m aam_o93 probes "sample" --modality text
python -m unittest discover -s tests -v
python scripts/verify_manifest.py
```

Les commandes exactes, l'environnement et le hash de version doivent être conservés dans le rapport correspondant. Les commandes réseau et capteurs nécessitent une autorisation distincte.

Chapitre 6

Conséquences et statut

6.1 Théoriques

L'implémentation met à l'épreuve la dépendance des empreintes à une famille de sondes. Les poids sont une hypothèse empirique versionnée, pas une vérité universelle pré-injectée.

6.2 Conceptuelles

La recherche classique est une compétence acquise parmi d'autres ; l'observation multi-sondes et les espaces sémantiques forment le chemin normal RAG. Une compétence est une procédure/mécanisme reproductible, composable et adaptable.

6.3 Opérationnelles

Les fonctions passent par un contrat commun et une mémoire causale ; toute preuve suit exactement la version testée. « Contourner une limitation » signifie rechercher un autre chemin permis (autre sonde, compétence, décomposition, ressource autorisée), jamais outrepasser consentement, sécurité, lois, permissions ou prétendre qu'un test manquant a réussi.

6.4 Conclusion/statut

Le formalisme spécifie la cible. Le statut réel reste celui des checklists et du registre de preuve. Aucun succès ni échec futur n'est supposé.

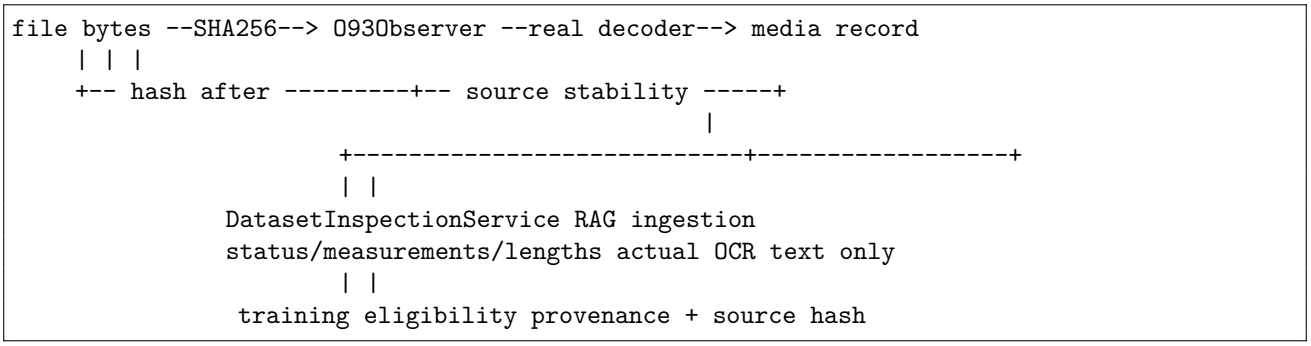
Chapitre 7

Dossier de validation P7 – lecteurs multimodaux réels

7.1 Description

Ce dossier ferme une lacune observée dans O93 : l'observateur classait l'image, l'audio et la vidéo principalement par empreintes, alors qu'un module de lecteurs bornés existait sans être appelé par le contrat central. Le patch P7 relie désormais les lecteurs au chemin `O93Observer.observe_path`, au service d'inspection de datasets et à l'ingestion RAG. Il n'assimile jamais des statistiques de pixels, des métriques de signal ou des frames décodées à une transcription ou à une compréhension de scène.

7.2 Architecture



7.3 Pipeline complet

1. Lire le fichier dans la limite de taille et calculer le SHA-256 des octets initiaux.
2. Calculer les vues d'empreinte à partir du snapshot brut sans exécuter le contenu.
3. Appeler le lecteur réel : Pillow pour les pixels ; SoundFile/libsndfile ou FFmpeg pour le PCM ; OpenCV pour des frames vidéo échantillonnées ; Tesseract seulement si disponible et autorisé par la politique locale.
4. Comparer le hash déclaré par le lecteur au hash initial et refaire le hash du chemin après observation. Toute divergence dégrade le statut et ne peut devenir un succès.
5. Exposer les mesures et le statut de décodage dans le rapport dataset sans y écrire le texte OCR brut.
6. Ajouter au RAG seulement le texte effectivement retourné par OCR image/frames vidéo. Chaque chunk garde le SHA-256 source, le type et le moteur d'extraction ; le contenu est non fiable et les drapeaux d'injection restent appliqués.

7. Pour l'audio, ne créer aucun transcript si aucun moteur ASR réel n'est configuré. Le PCM et ses mesures peuvent être observés sans rendre le fichier éligible à une acquisition textuelle supervisée.

7.4 Contrats de statut

Modalité	Statut borné observé	Interprétation permise
Image	`PIXELS_DECODED` ; `IMAGE_OCR_TEXT_DECODED` ou `IMAGE_PIXELS_DECODED_NO_OCR_TEXT`	Dimensions, statistiques pixels et hash sont des observations ; seul le texte réellement renvoyé par OCR est du texte extrait.
Audio	`PCM_DECODED` ; `AUDIO_PCM_DECODED_ASR_NOT_CONFIGURED`	Mesures de signal uniquement ; `ASR_STATUS=ENGINE_NOT_CONFIGURED` ne constitue pas une transcription.
Vidéo	`VIDEO_FRAMES_DECODED` ; `VIDEO_FRAMES_OCR_TEXT_DECODED_NO_ASR` ou `VIDEO_FRAMES_DECODED_ASR_NOT_CONFIGURED`	Frames réellement décodées et éventuellement OCR par frame ; aucun récit de scène ni transcription sonore n'est inféré.
Parquet	lecteur natif lorsqu'il existe, sinon `o930_parquet_flat_fallback`	Le fallback est limité aux schémas primitifs plats et aux encodages/compresseurs supportés ; une fonction non supportée donne `NOT_DECODED`, un fichier invalide donne `ERROR`.

7.5 Commandes exactes d'exécution

Depuis `Dmodel_AAM_O93_0` :

```
PYTHONPATH=. python -m pytest -q tests/test_p7_dataset_service.py
PYTHONPATH=. python -m pytest -q tests/test_p7_dataset_service.py tests/test_p7_integrated.
    py tests/test_p7_training_control.py
```

Les commandes réellement exécutées et leur variable `PYTHONPATH` absolue sont conservées dans les prévols. Les fichiers stdout, stderr, codes retour et SHA-256 sont associés aux résultats `O930_P7_media_decoder_result.json` et aux journaux individuels.

7.6 Environnement et intégrité

Le moteur local ne dispose pas de PyArrow/Fastparquet/Polars/DuckDB ; le fallback Parquet existant a néanmoins décodé le fichier source `source_material/dataset/0000.parquet` en lecture seule : 7 473 lignes, colonnes `question`/`answer`, 8 groupes de lignes.

Le SHA-256 du fichier Parquet est affiché ci-dessous en deux lignes successives, à concaténer sans séparateur :

```
ea82612ea9582142387730c793eb67d3
b12849002bc0b7fa6f8efafa7351419d
```

Le SHA-256 de l'archive source originale est affiché ci-dessous en deux lignes successives, à concaténer sans séparateur :

```
add72728952cd4ec83b6994a02d28022
339fbbed29835d40c01b8f26bca60c38
```

Les tests médias créent les images, signaux et vidéos dans des répertoires temporaires, comparent leurs hashes avant/après et ne réécrivent pas les sources.

Le SHA-256 du fichier Parquet est donné ci-dessous en deux lignes successives, à concaténer sans séparateur :

```
ea82612ea9582142387730c793eb67d3
b12849002bc0b7fa6f8efafa7351419d
```

Le SHA-256 de l'archive source originale est lui aussi donné en deux lignes successives, à concaténer sans séparateur :

```
add72728952cd4ec83b6994a02d28022
339fbbed29835d40c01b8f26bca60c38
```

Les tests médias créent les images, signaux et vidéos dans des répertoires temporaires, comparent leurs hashes avant/après et ne réécrivent pas les sources.

7.7 Résultats bruts

- `media_decoder_dataset_service_pytest` : code 0, 18 tests réussis, 4 sous-tests, 5.87 s ; stderr vide.
- `media_decoder_cross_integration_pytest` : code 0, 27 tests réussis, 9 sous-tests, 16.74 s ; stderr vide.
- `runtime_parquet_inspection_tests` : code 0, 20 tests réussis, 9 sous-tests ; inspection runtime non supervisée des 7 473 enregistrements sans clé cible inventée.

Les résultats complets, sorties et hashes sont consignés dans `Dmodel_AAM_O93_0/logs/`. Le test OCR/RAG prouve qu'un vrai texte retourné par Tesseract est ingéré avec provenance `kind=ocr` ; il ne prouve pas une compréhension générale d'image.

7.8 Interprétation des résultats

Les lecteurs image, audio et vidéo sont maintenant exécutés et leurs mesures remontent par l'observateur central. Le service dataset distingue `observation_eligible` de `training_eligible` ; des pixels seuls ou du PCM sans transcript ne sont pas revendiqués comme contenu textuel entraînable. L'ingestion RAG ne génère pas de légende ni de transcription artificielle. Les résultats sont bornés aux formats et budgets testés.

7.9 Conséquences

Théoriques : l'identité par octets et la mesure d'une modalité restent distinctes de l'équivalence sémantique. **Conceptuelles :** l'observation produit plusieurs niveaux, mais un niveau bas n'est pas promu en interprétation. **Opérationnelles :** les rapports exposent lecteur, mesures, provenance, OCR/ASR et limites ; un moteur absent ou une structure non prise en charge reste explicite.

7.10 Erreurs, limites et conclusion/statut

Le moteur ASR reste `ENGINE_NOT_CONFIGURED` ; l'audio ne produit donc pas de transcript. Le fallback Parquet ne remplace pas un moteur général et ne couvre pas les fonctions Parquet hors contrat. Les périphériques physiques, les connecteurs réseau et l'acceptation Windows native ne sont pas validés par ces tests. Statut de phase : la checklist et le registre de preuves sont les sources autoritatives pour l'état courant ; les limites ASR, Parquet général, capteurs physiques, réseau distant et validation Windows restent explicites.

7.11 Anomalies documentaires et retests

Compilation initiale. Le premier essai a échoué sur le chemin du lecteur d'observation. Le diagnostic conservé dans le journal était :

```
Missing $ inserted
O93Observer.observe_path
```

Le balisage inline a été corrigé, puis deux passes réussies ont confirmé la réparation.

Inspection visuelle. Le premier rendu montrait un chevauchement des statuts dans le tableau des modalités, ainsi que des hashes qui dépassaient la marge. **Correction.** Les colonnes sont contraintes à la largeur utile, les identifiants longs ont des points de coupure et les SHA-256 sont affichés en deux lignes successives à concaténer sans séparateur. **Incident d'inspection.** Une commande de lecture interrompue par `SIGPIPE` a été remplacée par une lecture directe des journaux JSON. Aucune archive ni source dataset n'a été modifiée.

7.12 Réconciliation finale de P7 – lecteurs, entraînement et preuves

7.12.1 Description et architecture

Le contrat central d'observation est raccordé aux lecteurs réels bornés de texte, documents, image, audio et vidéo. Le service dataset sépare l'observation, l'éligibilité à l'entraînement et la disponibilité du texte dérivé. Les exemples ne sont ajoutés au RAG qu'après extraction réelle et avec provenance ; aucune transcription ASR n'est inventée.

7.12.2 Pipeline exécuté

1. Relire formalisme, laboratoire, checklist de phases et deux checklists d'erreurs ; vérifier le hash source et le préflight.
2. Exécuter les matrices isolées et les tests dataset/média/Parquet ; conserver sorties, erreurs, codes retour et SHA-256.
3. Exécuter le smoke de l'interface Entraînement ; constater inspection, pause à la frontière de gradient, checkpoint, reprise, capsule et arrêt.
4. Compiler chaque document LaTeX en deux passes, synchroniser les PDF, rendre les pages et inspecter les tableaux, statuts, chemins et hashes.
5. Comparer les hashes binaires des copies des PDF et exécuter le préflight PDF avant la réconciliation des gates.

7.12.3 Commandes de référence réellement exécutées

```
python -m pytest -q tests/test_p7_dataset_service.py tests/test_p7_integrated.py tests/
    test_p7_training_control.py
python -m pytest -q tests/test_p7_dataset_service.py
pdflatex -interaction=nonstopmode -halt-on-error -file-line-error laboratoire_093_0.tex
pdflatex -interaction=nonstopmode -halt-on-error -file-line-error formalisme_093_0.tex
python /home/oai/skills/pdfs/scripts/pdf_preflight.py laboratoire_093_0.pdf --json
python /home/oai/skills/pdfs/scripts/pdf_preflight.py formalisme_093_0.pdf --json
```

7.12.4 Résultats bruts

— Matrice pytest historique isolée : 29 modules, 206 tests réussis, 0 échec. Elle constitue un lot de base versionné ; elle n'est pas additionnée aux tests P7 plus récents.

— Régression P7 courante : 27 tests et 9 sous-tests réussis ; la suite dataset/média dédiée compte 18 tests et 4 sous-tests ; une suite spécifique d'inspection Parquet compte 20 tests et 9 sous-tests.

— Intégration média : images décodées par Pillow ; mesures PCM audio par SoundFile/FFmpeg selon disponibilité ; frames vidéo par OpenCV ; tout OCR n'est déclaré qu'après retour réel du moteur configuré.

— Dataset Parquet source décodé par le fallback limité : 7 473 lignes, colonnes `question`/`answer`, 8 row groups. Hash source identique avant/après.

— Smoke de l'interface Entraînement : 24 exemples inspectés, pause/checkpoint, reprise, ajustement de 120 époques sur le dataset temporaire, capsule/checkpoint hachés et arrêt propre. Source temporaire immuable.

— PDF finaux : laboratoire 17 pages ; formalisme 17 pages. Le préflight de chaque PDF confirme un fichier ouvrable, non chiffré et sans avertissement. Les rendus visuels vérifiés sont archivés dans les sous-dossiers `evidence/` du paquet Dmodel ; les manifestes de rendu conservent les chemins exacts.

Les familles de tests se recouvrent ; leurs nombres ne doivent pas être additionnés comme s'il s'agissait d'échantillons indépendants.

7.12.5 Interprétation des résultats

Ces preuves établissent le décodage et le pilotage réellement testés dans l'environnement local et les budgets annoncés. Elles ne démontrent pas une compréhension générale de tous les contenus multimodaux, ni que tout producteur Parquet est pris en charge. L'adaptateur `o930_parquet_flat_fallback` n'est pas un moteur Parquet général.

7.12.6 Conséquences théoriques, conceptuelles et opérationnelles

Théoriques : distinguer l'empreinte des octets, les mesures de modalité, les textes OCR et une hypothèse de compréhension. Conceptuelles : l'éligibilité d'une donnée à l'observation n'est pas la preuve qu'elle forme un exemple textuel d'entraînement. Opérationnelles : le checkpoint, le lecteur, la provenance, le hash source et le statut de décodage sont conservés et vérifiables.

7.12.7 Erreurs, limites et statut

Le moteur ASR reste `ENGINE_NOT_CONFIGURED` ; aucun transcript audio n'est produit. Le fallback Parquet ne couvre que les schémas primitifs plats et codecs testés. Capteurs physiques, réseau distant/MCP, acceptation Windows native et entraînement à grande échelle restent `NOT_RUN`. La décision administrative de gate est consignée dans la checklist, le registre et le reçu de transition ; la complétude globale du modèle n'est pas revendiquée.